



Python Class Note: Operators

1. Introduction to Operators

Operators are special symbols in Python that perform operations on variables and values. They are like the “verbs” of programming — they let you add, compare, assign, and manipulate data.

2. Types of Operators in Python

A. Arithmetic Operators

Used for mathematical operations.

Operator	Meaning	Example (a=10, b=3)	Result
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division	a / b	3.33
//	Floor Division	a // b	3
%	Modulus (Remainder)	a % b	1
**	Exponentiation	a ** b	1000



Hands-on Practice (Basic):

```
a = 15
b = 4

print("Addition:", a + b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Remainder:", a % b)
print("Power:", a ** b)
```

B. Relational / Comparison Operators

Used to compare values. Returns `True` or `False` .

Operator	Meaning	Example (<code>a=5</code> , <code>b=10</code>)	Result
<code>==</code>	Equal to	<code>a == b</code>	<code>False</code>
<code>!=</code>	Not equal to	<code>a != b</code>	<code>True</code>
<code>></code>	Greater than	<code>a > b</code>	<code>False</code>
<code><</code>	Less than	<code>a < b</code>	<code>True</code>
<code>>=</code>	Greater than or equal	<code>a >= b</code>	<code>False</code>
<code><=</code>	Less than or equal	<code>a <= b</code>	<code>True</code>

 Hands-on Practice (Intermediate):

```
x = 7
y = 10
```

```
print(x > y)
print(x == y)
print(x != y)
```

C. Logical Operators

Used to combine conditional statements.

Operator	Meaning	Example	Result
<code>and</code>	Returns <code>True</code> if both are <code>True</code>	<code>(x > 5 and y > 5)</code>	<code>True</code>
<code>or</code>	Returns <code>True</code> if at least one is <code>True</code>	<code>(x > 5 or y < 5)</code>	<code>True</code>
<code>not</code>	Reverses the result	<code>not(x > 5)</code>	<code>False</code>

 Hands-on Practice (Intermediate):

```
age = 20
country = "Nigeria"

print(age > 18 and country == "Nigeria") # True
print(age < 18 or country == "Ghana")    # False
print(not(age > 18))                     # False
```

D. Assignment Operators

Used to assign values to variables (with shortcuts).

Operator	Meaning	Example (x=5)	Result
=	Assign	x = 5	5
+=	Add & Assign	x += 3	8
-=	Subtract & Assign	x -= 2	6
*=	Multiply & Assign	x *= 2	12
/=	Divide & Assign	x /= 2	6.0
%=	Modulus & Assign	x %= 4	2
**=	Exponent & Assign	x **= 2	4
//=	Floor divide & Assign	x //= 2	2

👉 Hands-on Practice (Basic):

```
x = 10
x += 5
print("x after += 5:", x)

x *= 2
print("x after *= 2:", x)
```

E. Bitwise Operators

Work on bits (0 and 1). Mostly used in system-level programming.

Operator	Meaning	Example (a=6(110), b=3(011))	Result		
&	AND	a & b	2 (010)		
`	`	OR	`a	b`	7 (111)
^	XOR	a ^ b	5 (101)		
~	NOT	~a	-7		
<<	Left Shift	a << 1	12		
>>	Right Shift	a >> 1	3		

👉 Hands-on Practice (Advanced):

```
a = 6 # binary: 110
b = 3 # binary: 011

print("a & b:", a & b)
print("a | b:", a | b)
print("a ^ b:", a ^ b)
print("a << 1:", a << 1)
print("a >> 1:", a >> 1)
```

F. Membership Operators

Used to check membership in sequences (lists, strings, tuples).

Operator	Meaning	Example	Result
in	Returns True if found	'a' in 'apple'	True
not in	Returns True if not found	'b' not in 'apple'	True

👉 Hands-on Practice:

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)
print("grape" not in fruits)
```

G. Identity Operators

Used to compare memory locations (whether two objects are the same).

Operator	Meaning	Example	Result
is	Returns True if both refer to same object	x is y	True/False
is not	Returns True if not same object	x is not y	True/False

👉 Hands-on Practice:

```
x = [1,2,3]
y = x
z = [1,2,3]

print(x is y)      # True (same object)
print(x is z)      # False (different objects with same values)
print(x == z)      # True (same values)
```

3. Real Life Example

```
# Checking if a student qualifies for scholarship
score = 85
age = 19

if score >= 80 and age < 21:
    print("Eligible for scholarship")
else:
    print("Not eligible")
```

4. Classwork Exercises

● Basic

1. Calculate the area and perimeter of a rectangle using arithmetic operators.
2. Check if a number is even or odd using % .

Intermediate

3. Write a program to check if a number is within a given range (10–50).
4. Simulate login logic: user must enter correct username AND password.

Advanced

5. Using bitwise operators, swap two numbers without using a third variable.
6. Use membership operators to check if a word exists inside a sentence.